# Practical 4

Part A — Practice Programs (Hour 1)Work these in your lab-NN/ folder before the project: do the first one or two in the lab; the last (For Advanced Learners) is for home.

## 1) Password strength (PasswordChecker.java + Driver.java)

(a) In Password Checker, check four rules — length ≥ 8, contains an uppercase letter, contains a digit, contains a special character — using regex (e.g. ".*[A-Z].*") or String methods.

(b) Write strength(String pw) that counts how many rules pass and returns "Weak" (0– 1), "Medium" (2–3) or "Strong" (4).

(c) In Driver .main: test several passwords; for each, print which rules passed and the final label. Expected output: "abc" → Weak; "Abcd1234!" → Strong.2.

Sol)

**Passwordchaecker.java**

```
public class PasswordChecker {

   public static boolean hasMinimumLength(String pw) {
      return pw.length() >= 8;
   }

   public static boolean hasUppercase(String pw) {
      return pw.matches(".*[A-Z].*");
   }

   public static boolean hasDigit(String pw) {
      return pw.matches(".*[0-9].*");
   }

   public static boolean hasSpecialCharacter(String pw) {
      return pw.matches(".*[^a-zA-Z0-9].*");
   }

   public static String strength(String pw) {
      int count = 0;

      if (hasMinimumLength(pw))
         count++;

      if (hasUppercase(pw))
         count++;

      if (hasDigit(pw))
         count++;

      if (hasSpecialCharacter(pw))
```

```
            count++;

        if (count <= 1)
            return "Weak";
        else if (count <= 3)
            return "Medium";
        else
            return "Strong";
    }
}
```

## Driver.java

```
public class Driver {

    public static void main(String[] args) {

        String[] passwords = {
            "abc",
            "abcdefgh",
            "Abcdefgh",
            "Abcd1234",
            "Abcd1234!"
        };

        for (String pw : passwords) {

            System.out.println("Password: " + pw);

            System.out.println("Length >= 8: "
                    + PasswordChecker.hasMinimumLength(pw));

            System.out.println("Uppercase: "
                    + PasswordChecker.hasUppercase(pw));

            System.out.println("Digit: "
                    + PasswordChecker.hasDigit(pw));

            System.out.println("Special character: "
                    + PasswordChecker.hasSpecialCharacter(pw));

            System.out.println("Strength: "
                    + PasswordChecker.strength(pw));

            System.out.println();
        }
    }
}
```

## Expected Output

```
Password: abc
Length >= 8: false
Uppercase: false
Digit: false
```

Special character: false
Strength: Weak

Password: abcdefgh
Length >= 8: true
Uppercase: false
Digit: false
Special character: false
Strength: Weak

Password: Abcdefgh
Length >= 8: true
Uppercase: true
Digit: false
Special character: false
Strength: Medium

Password: Abcd1234
Length >= 8: true
Uppercase: true
Digit: true
Special character: false
Strength: Medium

Password: Abcd1234!
Length >= 8: true
Uppercase: true
Digit: true
Special character: true
Strength: Strong

## Laerning Outcome

- Understand how to validate strings using **regular expressions**.
- Learn to count how many password rules are satisfied.
- Practice **methods, boolean conditions, and if-else statements**.
- Understand how multiple validation rules can be combined to classify input as **Weak, Medium, or Strong**.

2) Chat-log filter (ChatFilter.java + Driver.java)

(a) In Driver.main, define a String[] of log lines like “10:05 alice Hello there” (include one malformed line with fewer than 3 parts).

(b) Read a keyword from the user.

(c) For each line, split into time/user/message with split(“ ”, 3); skip lines that do not have 3 parts.

(d) If the message contains the keyword (case-insensitive), increment a counter and append “time user: message” to a StringBuilder.

(e) Print “Matches: N” and the built report.Expected output: keyword “hello” over 3 lines (1 malformed) → “Matches: 1” and the matching line.3

Sol)

**ChatFilter.java**

```
public class ChatFilter {

    public static String filterLogs(String[] logs, String keyword) {

        int count = 0;
        StringBuilder report = new StringBuilder();

        for (String line : logs) {

            String[] parts = line.split(" ", 3);

            if (parts.length < 3) {
                continue;
            }

            String time = parts[0];
            String user = parts[1];
            String message = parts[2];

            if (message.toLowerCase().contains(keyword.toLowerCase())) {

                count++;

                report.append(time)
                      .append(" ")
                      .append(user)
                      .append(": ")
                      .append(message)
                      .append("\n");
            }
        }

        return "Matches: " + count + "\n" + report;
    }
}
```

**Driver.java.**

```
import java.util.Scanner;

public class Driver {

    public static void main(String[] args) {

        String[] logs = {
            "10:05 alice Hello there",
            "10:10 bob Good morning",
            "10:15 Charlie hello everyone",
            "10:20"
        };
```

```
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter keyword: ");
        String keyword = sc.nextLine();

        String result = ChatFilter.filterLogs(logs, keyword);

        System.out.println(result);

        sc.close();
    }
}
```

## Expected Output

Enter keyword: hello
Matches: 2
10:05 alice: Hello there
10:15 Charlie: hello everyone

## Lerning Outcome

Understand how to process **arrays of strings**.
Practice `split(" ", 3)` for separating log information.
Learn to handle **malformed input** safely.
Understand **case-insensitive string searching**.
Practice using `StringBuilder` to construct a report.
Learn how to maintain a **counter while processing data**.

3)

(For Advanced Learners – home) Template filler (TemplateFiller.java +Driver.java)

(a) In Driver.main, define a template like “Dear {name}, order {id} ships {date}.” and two parallel arrays: names {“name”, “id”} and values {“Riya”, “A07”} (note {date} is not supplied).

(b) Compile a Pattern for \{(\w+)\} and get a Matcher on the template.

(c) Walk the matches with a StringBuilder: for each {placeholder}, look up its value in the names array and replace with the value, or with “[?]” if not found.

(d) Print the filled template.Expected output: the unsupplied {date} becomes [?]

Sol)

## TemplateFIller.java

```
import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class TemplateFiller {

    public static String fillTemplate(
            String template,
            String[] names,
```

```
            String[] values) {

        Pattern pattern = Pattern.compile("\\{(\\w+)\\}");
        Matcher matcher = pattern.matcher(template);

        StringBuilder result = new StringBuilder();

        int lastPosition = 0;

        while (matcher.find()) {

            result.append(template, lastPosition, matcher.start());

            String placeholder = matcher.group(1);

            String replacement = "[?]";

            for (int i = 0; i < names.length; i++) {

                if (names[i].equals(placeholder)) {
                    replacement = values[i];
                    break;
                }
            }

            result.append(replacement);

            lastPosition = matcher.end();
        }

        result.append(template.substring(lastPosition));

        return result.toString();
    }
}
```

**Driver.java**

```
public class Driver {

    public static void main(String[] args) {

        String template =
                "Dear {name}, order {id} ships {date}.";

        String[] names = {
            "name",
            "id"
        };

        String[] values = {
            "Riya",
            "A07"
        };

        String result = TemplateFiller.fillTemplate(
                template,
                names,
```

```
            values
        );

        System.out.println(result);
    }
}
```

## Expected Output

Dear Riya, order A07 ships [?]..

## Learning Outcome

Understand **Pattern and Matcher** in Java.
Practice using **regular expressions** to identify placeholders.
Learn how `Matcher.find()` and `matcher.group()` work.
Practice using `StringBuilder` to construct modified strings.
Understand how to search **parallel arrays** for corresponding values.
Learn how to provide a default value such as `[?]` when a placeholder is not found.